第12章运行说明

返回正文

Java开发工具包（Java Development Kit，JDK）25.0.5.0.2、Maven3.10.0、macOS AArch64。普通教学与有界观测随用户“执行P04”授权；没有测试、断言、故障耗尽、压力或性能基准。正文与源阅读副本为chapter.html、sources.html。

示例	源码 / 对应节 / 观察目的	资源和结束
CH12-E01 NativeJourney	src/main/java/org/example/jvminternals/ch12/NativeJourney.java；12.1—12.5；基线、持有与释放的用途账本	每阶段45秒，约135秒自然退出；4工作线程在主流程停止后join，自身最多等待90秒；8MiB直接缓冲区、256KiB外部块、24加载器；最多两次回收请求

MiB为兆二进制字节（mebibyte），KiB为千二进制字节（kibibyte）。堆32—128MiB，每教学平台线程栈512KiB，直接池容量上限16MiB。只保留一个容量1的直接切片；Arena显式关闭，直接池清理与类卸载依赖回收协作。不访问失效内存，不创建网络连接。提前停止只中断自己的新进程；最终系统会回收进程资源，但这不代替长进程及时关闭。

编译与运行

工作目录为项目根目录。Disposable明确复用CH08源，单独编译到本章target/disposable；它不在主类路径，不依赖其他章先编译。

```
task_jdk=/opt/homebrew/Cellar/openjdk@25/25.0.5.0.2/libexec/openjdk.jdk/Contents/Home
JAVA_HOME="$task_jdk" mvn -B -ntp -pl chapters/ch12-native-memory -am compile
"$task_jdk/bin/javac" --release 25 -g -parameters -d chapters/ch12-native-
memory/target/disposable chapters/ch08-type-
metadata/variants/disposable/org/example/jvminternals/ch08/disposable/Disposable.java
"$task_jdk/bin/java" -Xms32m -Xmx128m -Xss512k -XX:MaxDirectMemorySize=16m -
XX:NativeMemoryTracking=summary -
Xlog:class+unload=info,gc+metaspace=info,gc=info:file=chapters/ch12-native-
memory/evidence/runs/ch12-20261103-02/lifecycle.log -cp chapters/ch12-native-
memory/target/classes org.example.jvminternals.ch12.NativeJourney chapters/ch12-native-
memory/target/disposable 2>&1 | tee chapters/ch12-native-memory/evidence/runs/ch12-
20261103-02/program.txt
```

本地内存跟踪（Native Memory Tracking，NMT）必须从启动启用，附加需本机诊断权限。观察需另一个终端；先从程序输出取得新进程号，不重用本轮历史编号23682。文件名在重放时选择新运行目录，不覆盖本

轮证据。

三阶段采集命令

设置task_pid为自己刚启动进程的编号。每次先确认程序阶段输出，再保存采集开始与结束时间。以下是本轮调用形式；完整三组输出与时间见运行记录。

基线阶段：

```
"$task_jdk/bin/jcmd" "$task_pid" VM.native_memory baseline
"$task_jdk/bin/jcmd" "$task_pid" VM.native_memory summary scale=KB
"$task_jdk/bin/jcmd" "$task_pid" GC.heap_info
"$task_jdk/bin/jcmd" "$task_pid" VM.classloader_stats
ps -p "$task_pid" -o pid,rss,vsz,etime,command
vmmap -summary "$task_pid"
```

持有阶段与释放请求后阶段分别运行下组；两次都与同一个初始NMT基线比较，不在中间重新baseline：

```
"$task_jdk/bin/jcmd" "$task_pid" VM.native_memory summary.diff scale=KB
"$task_jdk/bin/jcmd" "$task_pid" GC.heap_info
"$task_jdk/bin/jcmd" "$task_pid" VM.classloader_stats
ps -p "$task_pid" -o pid,rss,vsz,etime,command
vmmap -summary "$task_pid"
```

持有阶段还执行一次 `"$task_jdk/bin/jcmd" "$task_pid" Thread.print`。两阶段的文件前缀分别held与released；基线为base。NMT内部类别、Java管理接口和系统线程 / 驻留口径不同，不要求数值逐项相同。

预期观察的是持有链与用途变化，不预设某个回收时点、卸载数量或驻留回落。首轮15秒窗口的外部持有快照采晚，单独保留为不足证据；本轮窗口和真实阶段已对应。退出、当前输入身份与全部原始输出位置以运行记录为准。